DBMS 동시성 및 설계 가이드

정보 전달을 최우선으로 두고, 읽는 흐름이 자연스럽도록 PDF로 재구성한 문서

1. 문서 목적

이 문서는 DBMS를 만들 때 부딪히는 동시성 문제와, 그 외에 반드시 함께 설계해야 하는 핵심 요소를 한 번에 정리한 문서다.

특히 아래 세 층위를 분리해서 보는 것이 중요하다.

1. 멀티스레드 동기화: 엔진 내부의 공유 메모리와 자료구조를 안전하게 보호하는 문제
2. 트랜잭션 동시성 제어: 여러 사용자 요청이 동시에 들어와도 데이터 의미를 올바르게 유지하는 문제
3. 장애 복구/분산 일관성: 크래시, 디스크 flush 순서, 복제, 리더 장애 상황에서도 일관성을 유지하는 문제

이 세 가지를 섞어서 설계하면 구현이 복잡해지고, 성능 문제와 정합성 문제가 동시에 생기기 쉽다.

읽는 순서

이 문서는 한 번에 처음부터 끝까지 읽어도 되지만, 목적에 따라 골라 읽는 편이 더 잘 들어온다.

읽는 목적	먼저 보면 좋은 장	이유
큰 그림 이해	2, 3, 4, 5, 6, 7	동시성 문제가 어떤 층위로 나뉘는지 먼저 잡을 수 있다
구현 판단	8, 9, 10, 11, 14, 15, 16	인덱스, 스토리지, 복구, 실행 엔진, 현실적인 구현 선택을 바로 연결할 수 있다
현재 프로젝트 적용	14, 15, 16, 17	지금 레포에 무엇을 먼저 넣고 무엇은 나중에 미뤄야 하는지 판단할 수 있다
심화 학습	부록 A, 부록 B	MVCC 내부 구조, B+Tree latch, WAL 레코드, 제품 사례까지 더 깊게 볼 수 있다

문서 구성

이 문서는 아래 흐름으로 읽히도록 구성되어 있다.

1. 2-13장: 핵심 본문. 동시성, 트랜잭션, 인덱스, 스토리지, 복구, 분산, 운영의 기본 구조를 설명한다.
2. 14-17장: 구현 판단과 현재 프로젝트 적용. 지금 당장 무엇을 선택하고 무엇을 미뤄야 하는지 정리한다.
3. 부록 A-B: 심화 주제와 추가 제품 설계 체크사항. 본문을 더 깊게 확장하는 참고 자료다.

2. 먼저 기억해야 할 핵심 원칙

1. 락을 더 정교하게 거는 것보다 공유 상태 자체를 줄이는 것이 더 큰 효과를 낸다.
2. 자료구조 보호용 latch와 트랜잭션 격리용 lock은 목적이 다르므로 분리해서 설계해야 한다.

3 읽기 성능, 쓰기 성능, 복구 단순성, 운영 복잡도는 서로 교환 관계가 있다.

4 DBMS는 평상시 성능보다 드물게 발생하는 실패 상황에서 얼마나 안전한가가 더 중요하다.

5 단일 노드에서 맞는 설계가 분산 환경에서 그대로 맞지 않는다.

3. 한눈에 보는 문제와 해결법

구분	대표 문제	대표 해결법	특징	장점	단점	주 적용 위치
엔진 내부 동기화	race condition, data corruption	`mutex`, `rwlock`, `spinlock`	가장 기본적인 임계구역 보호	구현 단순, 검증 쉬움	경합 시 대기 증가, deadlock 가능	버퍼 풀, 카탈로그 캐시, 큐
읽기 많은 구조	read-heavy contention	`rwlock`, `RCU`, copy-on-write	읽기와 쓰기 비용 비대칭	읽기 성능 좋음	writer starvation, 구현 복잡도 상승	메타데이터, 스키마 캐시
핫스팟	특정 키/버킷 경합	sharding, lock striping, per-core state	락 종류보다 경합 분산에 초점	스케일 아웃에 유리	메모리 증가, merge 비용	lock manager, 통계 카운터
트리 인덱스 수정	split/merge 시 충돌	latch coupling, optimistic latch, B-link tree	B+Tree 전용 기법	높은 동시성	corner case 많음	B+Tree 내부 노드
트랜잭션 격리	dirty read, lost update	`2PL`, `strict 2PL`	락 기반 전통 방식	직관적, 직렬화 보장 쉬움	블로킹, deadlock	OLTP, 메타데이터 락
읽기-쓰기 동시성	reader/writer 충돌	`MVCC`	버전 기반 snapshot 읽기	reader blocking 감소	vacuum/GC 필요	범용 RDBMS
충돌 드문 워크로드	불필요한 락 대기	`OCC`	커밋 시점 검증	짧은 트랜잭션에 유리	충돌 많으면 재시도 폭증	메모리 DB, 분산 KV
팬텀/쓰기 왜곡	phantom, write skew	predicate lock, next-key lock, `SSI`	단순 row lock보다 강력	serializable 구현 가능	구현/성능 비용 큼	높은 격리 수준
크래시 복구	torn write, partial commit	`WAL`, checkpoint, redo/undo	먼저 로그, 나중에 데이터	durability 핵심	복구 경로 설계 필요	거의 모든 DBMS
commit 지연	`fsync` 비용	group commit, batching	여러 트랜잭션 flush 병합	처리량 상승	소규모 트래픽에 이득 적음	OLTP 엔진
분산 일관성	리더 장애, 원자적 commit	`2PC`, Raft, Paxos	다중 노드 조정	강한 일관성 가능	느리고 복잡	분산 SQL, 메타데이터 서비스
lock-free 자료구조	높은 락 경합	CAS, hazard pointer, epoch GC	락 없이 진행 보장 시도	고경합에서 유리할 수 있음	ABA, 메모리 회수 난이도 큼	큐, freelist, 통계 구조

4. 멀티스레드에서 발생하는 대표 동시성 문제

4.1 Race Condition

두 스레드가 동일한 상태를 동시에 읽고 쓰면서 결과가 꼬이는 문제다.

예:

- 두 worker가 같은 `next_id`를 동시에 읽고 같은 값을 배정
- 같은 페이지의 dirty bit 또는 pin count를 동시에 갱신
- 같은 인덱스 노드를 동시에 split하면서 포인터 구조가 손상

해결:

- 가장 먼저 `mutex` 또는 `rwlock`으로 임계구역을 감싼다.
- 병목이 보이면 `coarse lock -> fine lock -> partitioned state` 순서로 개선한다.
- 숫자 카운터는 경우에 따라 `atomic`으로 대체할 수 있다.

장점:

- 단순하고 검증하기 쉽다.

단점:

- 락 범위를 잘못 잡으면 전체 처리량이 급락한다.

4.2 Deadlock

서로가 상대가 가진 락을 기다리면서 영원히 진행하지 못하는 문제다.

예:

- 트랜잭션 A는 row X를 잡고 row Y를 기다림
- 트랜잭션 B는 row Y를 잡고 row X를 기다림

해결:

- 락 획득 순서를 전역 규칙으로 고정
- `timeout` 또는 `deadlock detector` 도입
- `wait-die`, `wound-wait` 같은 예방 기법 사용

장점:

- 규칙만 잘 잡으면 예방 가능하다.

단점:

- fine-grained locking으로 갈수록 분석 난도가 올라간다.

4.3 Livelock / Starvation

락은 풀리지만 특정 스레드나 트랜잭션이 계속 밀려 실제로는 일을 못 하는 문제다.

예:

- writer가 계속 들어오지 못하는 `rwlock`
- CAS 재시도가 끝없이 반복되는 lock-free 구조

해결:

- 공정 락 사용
- backoff 정책 추가
- queue-based lock이나 우선순위 정책 도입

4.4 Hotspot Contention

하나의 전역 락이나 특정 키에 요청이 몰려 병목이 생기는 문제다.

예:

- 전역 버퍼 풀 락
- 전역 lock manager hash bucket
- 단일 `next_id`

해결:

- lock striping
- shard별 분리
- per-thread / per-core 카운터
- single-writer actor 구조

핵심:

락을 더 빠르게 만드는 것보다 경합 지점을 쪼개는 것이 보통 더 효과적이다.

4.5 False Sharing

서로 다른 데이터지만 같은 CPU cache line 안에 있어서 성능이 급락하는 문제다.

예:

- worker별 통계 카운터가 연속 메모리에 붙어 있는 경우
- 큐 head/tail이 같은 cache line을 공유하는 경우

해결:

- cache line padding
- 자주 갱신되는 필드 분리
- read-mostly 구조와 write-hot 구조 분리

4.6 Memory Ordering / ABA / Memory Reclamation

lock-free 자료구조에서는 단순히 `CAS` 하나만 넣는다고 끝나지 않는다.

추가 문제:

- CPU 재정렬로 인해 의도와 다른 관측이 발생
- ABA로 인해 포인터가 바뀌었는데도 안 바뀐 것처럼 보임
- 다른 스레드가 아직 읽는 노드를 너무 빨리 해제

해결:

- `acquire/release` 메모리 오더 사용
- tagged pointer
- `hazard pointer`
- `epoch-based reclamation`

결론:

DBMS 핵심 경로에서 lock-free를 쓰려면 매우 강한 이유가 있어야 한다. 일반적인 프로젝트에서는 잘 설계된 락 구조가 더 안전하다.

---

5. 엔진 내부 동기화 기법 정리

5.1 Coarse-Grained Lock

엔진 전체나 테이블 전체를 하나의 큰 락으로 보호하는 방식이다.

특징:

- 구현이 가장 쉽다.
- correctness를 빨리 확보할 수 있다.
- 초기 단계의 프로젝트에 현실적이다.

장점:

- 디버깅이 쉽다.
- 데이터 손상 위험을 빨리 줄일 수 있다.

단점:

- 읽기/쓰기 동시성이 크게 제한된다.
- worker 수를 늘려도 성능이 잘 안 오른다.

어디에 적용되는가:

- MVP 단계
- 단일 테이블 학습용 DB
- 먼저 안정성을 확보해야 하는 API 서버

5.2 Fine-Grained Lock

테이블, 페이지, 레코드, 인덱스 노드 등 더 작은 단위로 락을 나누는 방식이다.

장점:

- 병렬성이 좋아진다.
- 특정 테이블이나 페이지 경합이 전체를 막지 않는다.

단점:

- deadlock 가능성이 커진다.
- 락 계층과 순서를 엄격히 정의해야 한다.

적용:

- page latch
- row lock
- index node latch

5.3 Reader-Writer Lock

읽기는 동시에 허용하고, 쓰기만 배타적으로 처리하는 방식이다.

장점:

- 읽기 비중이 큰 워크로드에서 유리하다.

단점:

- writer starvation 가능
- 읽기 중에도 실제로는 쓰기처럼 동작하는 코드가 있으면 위험

적용:

- 카탈로그
- 스키마 캐시
- 현재 프로젝트처럼 `SELECT`와 `INSERT`를 거칠게 분리하는 단계

5.4 Spinlock

짧은 락 구간에서 잠들지 않고 바쁘게 도는 락이다.

장점:

- 매우 짧은 임계구역에서 context switch 비용이 적다.

단점:

- 락이 오래 잡히면 CPU를 낭비한다.

적용:

- 커널 수준
- 아주 짧은 메모리 구조 보호
- 일반적인 학습용 DBMS에서는 우선순위가 낮다

5.5 Lock Striping / Sharding

데이터를 여러 버킷으로 나누고 버킷마다 별도 락을 두는 방식이다.

장점:

- 전역 락보다 훨씬 잘 확장된다.

단점:

- resize, rehash, global operation이 복잡하다.

적용:

- hash index
- lock table
- buffer hash

5.6 Optimistic Latch / Version Check

일단 락 없이 읽고, 마지막에 버전이 안 바뀌었는지 확인하는 방식이다.

장점:

- 읽기 중심 구조에서 빠르다.

단점:

- 충돌이 많으면 재시도가 많아진다.
- 검증 실패 경로를 정확히 만들어야 한다.

적용:

- 인덱스 탐색
- read-mostly 메타데이터

5.7 Message Passing / Single-Writer

아예 공유 구조를 여러 스레드가 동시에 건드리지 않게 하고, 특정 shard는 오직 하나의 worker만 수정하게 만드는 방식이다.

장점:

- 락 복잡도가 급감한다.
- reasoning이 쉬워진다.

단점:

- 샤드 간 트랜잭션이 어려워진다.
- load imbalance가 생길 수 있다.

적용:

- actor 기반 저장소
- 파티션별 writer
- 로그 수집형 저장소

---

6. 트랜잭션 동시성 제어

엔진 내부 latch가 자료구조를 보호하는 것이라면, 트랜잭션 동시성 제어는 사용자 관점에서 데이터 의미를 보호하는 것이다.

6.1 막아야 할 대표 이상 현상

- `Dirty Read`: 커밋되지 않은 값을 읽음
- `Non-repeatable Read`: 같은 row를 다시 읽었더니 값이 바뀜
- `Phantom Read`: 같은 조건으로 다시 읽었더니 row 개수가 달라짐
- `Lost Update`: 두 트랜잭션의 갱신 중 하나가 사라짐
- `Write Skew`: 서로 다른 row를 읽고 검증한 뒤 각각 써서 제약이 깨짐

6.2 2PL

`Two-Phase Locking`은 락을 획득하는 단계와 해제하는 단계를 분리하는 전통적 방식이다.

특징:

- 직렬화 가능성 보장이 쉽다.
- row/table/predicate lock 개념과 잘 맞는다.

장점:

- 논리적으로 이해하기 쉽다.
- 전통적인 관계형 DB 이론과 잘 맞는다.

단점:

- 블로킹이 발생한다.
- deadlock detector가 필요하다.

적용:

- 전통적인 OLTP DB
- DDL/메타데이터 락

6.3 Strict 2PL

특히 write lock을 commit까지 유지한다.

장점:

- cascading abort를 막는다.
- 복구가 단순해진다.

단점:

- 대기 시간이 길어진다.

6.4 MVCC

한 레코드의 여러 버전을 유지하고, reader는 자기 snapshot에 맞는 버전을 읽는 방식이다.

특징:

- reader와 writer가 직접 충돌하지 않는 경우가 많다.
- 현대 범용 DBMS의 핵심 기법이다.

장점:

- 읽기 성능이 매우 좋다.
- 긴 조회가 짧은 쓰기를 덜 방해한다.

단점:

- `vacuum`, `undo purge`, old version GC가 필수다.
- 오래 열린 트랜잭션이 정리 작업을 막을 수 있다.
- storage bloat가 생길 수 있다.

실제 적용:

- PostgreSQL 계열
- InnoDB 계열

6.5 OCC

실행 중에는 락을 적게 쓰고, 커밋 직전에 읽은 데이터가 바뀌지 않았는지 검증한다.

장점:

- 충돌이 드문 짧은 트랜잭션에 유리하다.

단점:

- 충돌이 많으면 abort/retry 비용이 커진다.

실제 적용:

- 메모리 DB
- 분산 KV
- 짧은 read-modify-write

6.6 SSI

`Snapshot Isolation`만으로는 막히지 않는 `write skew`를 탐지하여 serializable에 가깝게 만드는 방식이다.

장점:

- MVCC 기반에서도 높은 격리 수준 제공 가능

단점:

- dependency tracking이 필요하다.
- 구현 복잡도가 높다.

6.7 Predicate Lock / Next-Key Lock

단순 row lock으로는 막을 수 없는 `phantom` 문제를 막기 위해 조건 범위를 잠그는 방식이다.

적용:

- `WHERE salary > 1000`
- `BETWEEN`
- 인덱스 범위 검색

---

7. 격리 수준과 선택 기준

격리 수준	막는 문제	보통 구현 방식	장점	단점	적합한 곳
`Read Uncommitted`	거의 없음	거의 사용 안 함	빠를 수 있음	dirty read 허용	일반 서비스에는 부적합
`Read Committed`	dirty read	read lock 짧게, MVCC snapshot	기본값으로 무난	repeatable read 아님	범용 API 서버
`Repeatable Read`	non-repeatable read	stronger lock 또는 snapshot	읽기 일관성 향상	phantom, write skew 이슈 가능	OLTP
`Snapshot Isolation`	dirty/non-repeatable read 상당수	MVCC	읽기 성능 좋음	write skew 가능	범용 RDBMS
`Serializable`	주요 이상 현상 대부분	strict 2PL, SSI, predicate lock	의미가 가장 명확	성능 비용 큼	금융, 강한 정합성 요구

실무 기준:

- 기본 API 서버는 `Read Committed` 또는 `Snapshot Isolation`부터 시작하는 경우가 많다.
- 강한 정합성이 필요하면 `Serializable`을 제공하되, 전체 기본값으로 두지는 않는 경우가 많다.

---

8. 인덱스와 자료구조 관점의 동시성

8.1 B+Tree

DBMS 인덱스의 대표 구조다.

장점:

- point lookup과 range scan 모두 강하다.
- 정렬 순서를 유지한다.
- 디스크 페이지 구조와 잘 맞는다.

동시성 문제:

- split/merge 시 부모 갱신
- leaf와 internal node의 일관성 유지
- 검색 중 구조 변경

대표 해결법:

- `latch coupling`: 부모를 잡은 상태에서 자식으로 내려감
- `optimistic read latch`: 읽기 후 버전 재검증
- `B-link tree`: sibling pointer와 high key로 동시 split 내성 강화

실제 적용:

- 전통적 RDBMS의 secondary index
- range query 중심 OLTP

8.2 Hash Index

장점:

- 단일 키 조회에 빠르다.

단점:

- range query가 약하다.
- rehash와 bucket split이 까다롭다.

동시성 포인트:

- bucket latch
- resize 시 global coordination

8.3 LSM Tree

쓰기 집약적 워크로드에 강한 구조다.

장점:

- 순차 쓰기 중심이라 ingest가 강하다.
- WAL + memtable + SSTable 구조가 단순하다.

단점:

- compaction 비용
- read amplification
- space amplification

실제 적용:

- 로그 저장소
- 시계열
- 대규모 쓰기 중심 KV

---

9. 스토리지 엔진에서 반드시 고려할 것

9.1 Page Layout

DB는 보통 `page` 단위로 읽고 쓴다.

중요 스펙:

- page size: 보통 `8KiB` 또는 `16KiB`
- `page header`
- `slot directory`
- variable-length record 관리
- free space offset

왜 중요한가:

- update, delete, compact, vacuum, checksum 설계의 기준이 된다.

9.2 Buffer Pool

디스크 페이지를 메모리에 캐시하는 계층이다.

핵심 요소:

- page table

- pin/unpin
- dirty flag
- eviction 정책

대표 정책:

- `Clock`
- `LRU-K`
- `2Q`

주의:

- 단순 `LRU`는 순차 스캔에 약하다.
- buffer pool 락이 전체 병목이 되기 쉽다.

9.3 Record Format

가변 길이 문자열, null bitmap, row header, version pointer 등을 어떻게 저장할지 미리 정해야 한다.

영향:

- MVCC 구현 방식
- index-only scan 가능 여부
- update in-place 가능 여부

---

10. 장애 복구와 durability

10.1 WAL

`Write-Ahead Logging`은 데이터 페이지를 디스크에 쓰기 전에, 변경 내역을 로그에 먼저 기록하는 방식이다.

핵심 규칙:

1. 데이터 페이지보다 로그가 먼저 durable해야 한다.
2. commit 응답은 필요한 로그가 flush된 뒤에만 내보내야 한다.

장점:

- 크래시 후 redo/undo가 가능하다.
- dirty page를 나중에 써도 된다.

단점:

- LSN 관리가 필요하다.
- 로그 형식 설계가 어렵다.

10.2 Checkpoint

모든 페이지를 즉시 플러시하지 않고, 특정 시점의 복구 기준점을 만든다.

장점:

- 재시작 시간이 짧아진다.

단점:

- checkpoint 순간 I/O 스파이크가 생길 수 있다.

10.3 Redo / Undo

- `redo`: 로그를 재적용해서 커밋된 변경을 반영
- `undo`: 커밋되지 않은 변경을 되돌림

선택:

- `undo+redo`
- `redo-only`
- append-only + compaction

10.4 Group Commit

여러 commit을 묶어 한 번에 `fsync`한다.

장점:

- 처리량이 크게 오른다.

단점:

- 단건 요청 지연은 약간 늘 수 있다.

10.5 추가 내구성 포인트

- page checksum
- WAL checksum
- torn write 대응
- crash injection 테스트
- `kill -9` 시뮬레이션

11. 쿼리 처리기와 실행 엔진

11.1 실행 모델

- `Volcano iterator`: 구현 단순, 교육용과 범용 엔진에 적합

- `Vectorized execution`: 분석 쿼리에 강함

11.2 병렬 실행

고려 요소:

- operator별 병렬화 가능 여부
- exchange operator
- work stealing
- data skew 대응

실제 문제:

- 병렬성이 높아도 전역 메모리 allocator나 결과 병합 단계에서 병목이 날 수 있다.

11.3 Optimizer

학습용 DB라도 최소한 아래는 필요하다.

- table scan vs index scan 선택
- predicate pushdown
- join order는 나중 단계라도 고려 대상

---

12. 분산 DBMS라면 추가로 생기는 문제

12.1 Replication

복제는 읽기 확장성과 장애 대응을 위해 필요하지만, replica lag가 곧 일관성 문제로 이어진다.

선택:

- sync replication
- async replication
- semi-sync replication

12.2 Consensus

리더 선출과 로그 순서 보장을 위해 `Raft` 또는 `Paxos`류가 필요하다.

장점:

- 리더 장애에도 일관된 의사결정 가능

단점:

- 단일 노드 DB보다 훨씬 복잡

- 네트워크 지연의 영향을 크게 받음

12.3 2PC

샤드나 노드가 여러 개인 트랜잭션에서 원자성을 보장하기 위한 대표 기법이다.

장점:

- 분산 commit 의미가 명확하다.

단점:

- coordinator 장애에 취약
- 느리다

12.4 시계 문제

분산에서는 시간 자체가 정합성 변수다.

필요한 기술:

- logical clock
- `HLC`
- clock uncertainty budget

---

13. 운영, 보안, 테스트

13.1 운영 지표

반드시 보고 있어야 할 지표:

- p50, p95, p99 latency
- queue 길이
- lock wait 시간
- deadlock count
- cache hit ratio
- WAL bytes/sec
- checkpoint stall
- compaction backlog
- replica lag

13.2 보안

- 인증

- 권한 관리
- audit log
- TLS
- at-rest encryption

13.3 백업/복원

- full backup
- incremental backup
- point-in-time recovery

13.4 테스트 전략

DBMS는 일반 단위 테스트만으로 충분하지 않다.

필수에 가까운 테스트:

- concurrency stress test
- fuzzing
- crash/recovery test
- long-running soak test
- deterministic scheduler 기반 race 재현
- Jepsen류 사고실험

14. 구현 단계별 현실적인 기술 선택

단계	추천 선택	이유	버려야 할 욕심
1단계 학습/MVP	전역 `mutex` 또는 테이블 단위 `rwlock`	correctness 확보가 가장 중요	처음부터 row lock, lock-free
2단계 API 서버 병렬화	bounded queue + thread pool + coarse DB lock	서버와 엔진 경계를 안정적으로 만들 수 있음	모든 worker가 동시에 write
3단계 읽기 확장	`rwlock`, snapshot read, metadata cache 분리	읽기 처리량 개선	복잡한 serializable 구현
4단계 실제 DB 구조화	WAL, checkpoint, buffer pool, page layout	데이터 안전성 확보	단순 파일 append만으로 충분하다는 가정
5단계 고성능화	fine-grained latch, lock striping, MVCC	병목 제거	근거 없는 lock-free 도입
6단계 분산화	replication, consensus, shard metadata	단일 노드 한계를 넘김	단일 노드 코드 재사용만으로 해결 가능하다는 기대

15. 현재 프로젝트에 바로 적용해서 보면

이 레포는 `task queue -> thread pool -> db_api -> executor/runtime/storage/bptree` 흐름을 가진다.

현재 구조를 기준으로 보면:

- `/src/task_queue.c`: `pthread_mutex`와 `pthread_cond`로 bounded queue를 보호한다.
- `/src/thread_pool.c`: 고정 worker들이 queue에서 작업을 꺼내 처리한다.
- `/src/db_api.c`: `pthread_rwlock_t` 하나로 `SELECT`와 `INSERT`의 동시 접근을 제어한다.

이 구조는 초기 correctness 확보용 coarse-grained concurrency control로는 적절하다.

현재 상태를 한 번에 묶어 보면 아래와 같다.

관점	현재 상태	장점	한계	다음 단계
서버 처리 구조	bounded queue + thread pool	구현 단순, backpressure 가능, worker 재사용 가능	queue/worker 수를 늘려도 DB 락이 병목이면 이득이 제한됨	queue 길이, worker utilization, latency를 먼저 측정
DB 접근 제어	`db_api.c`의 전역 `pthread_rwlock_t`	correctness 확보가 쉽고 테스트가 단순함	테이블/인덱스 내부 동시성이 전부 하나의 큰 락에 묶임	`global -> table -> page/latch` 순서로 점진 분리
엔진 구조	executor/runtime/storage/bptree가 하나의 coarse 정책 아래 동작	서버 레이어와 엔진 레이어 경계가 명확함	B+Tree 자체의 동시성 성능은 아직 확인 불가	storage/index latch와 transaction lock을 분리 설계
durability/운영성	아직 본격적인 WAL/복구/관측 체계는 없음	지금 단계에선 단순함 유지 가능	crash safety와 운영 복구 가능성이 약함	WAL 개념, write durability 규칙, 관측 지표를 먼저 문서화/구현

15.1 현재 방식의 장점

- 구현이 단순하다.
- data corruption 가능성을 빠르게 줄인다.
- 테스트와 디버깅이 쉽다.
- 서버 레이어와 엔진 레이어의 경계가 명확하다.

15.2 현재 방식의 한계

- 모든 테이블과 인덱스가 사실상 하나의 큰 락 정책 아래에 있다.
- write가 들어오면 읽기 병렬성이 크게 제한될 수 있다.
- B+Tree 내부 동시성은 전역 락이 대신 막아 주고 있으므로, 구조 자체의 동시성 성능은 아직 확보되지 않았다.

15.3 이 프로젝트에서 현실적인 개선 순서

1. 지금 단계에서는 전역 `rwlock` 정책을 유지하면서 correctness와 테스트를 먼저 고정한다.
2. 그다음 테이블 단위 락 또는 runtime metadata 락과 storage/index 락을 분리한다.
3. 이후 B+Tree page latch와 transaction lock을 분리 설계한다.

4 그 뒤에야 MVCC나 finer-grained row/page locking을 검토한다.

15.4 지금 이 프로젝트에서 추천하는 최소 기술 스펙

- worker pool 유지
- bounded queue 유지
- queue full 시 backpressure 유지
- DB 접근은 최소한 `table` 또는 `global` 단위 락 유지
- `WAL`은 아직 없더라도 write path 원자성 규칙은 미리 정의
- 향후를 위해 `page`, `LSN`, `checkpoint` 개념을 문서로 먼저 고정
- lock wait, queue length, request latency 지표 추가

즉, 현재 프로젝트에서는 멀티스레드 서버 병렬성과 DB 엔진 내부 정합성을 동시에 욕심내기보다, 먼저 안전한 전역 정책으로 시작하고 점진적으로 세분화하는 것이 맞다.

16. DBMS를 만들 때 빠뜨리기 쉬운 체크리스트

16.1 동시성

- 락 순서가 문서로 정의되어 있는가
- reader/writer 정책이 starvation을 유발하지 않는가
- queue와 worker 수를 늘렸을 때 병목 지점이 어디인지 보이는가

16.2 저장 구조

- page 크기가 정해져 있는가
- row format이 가변 길이를 지원하는가
- checksum과 file format version이 있는가

16.3 복구

- commit 전에 무엇이 flush되어야 하는가
- crash 후 redo/undo 경로가 있는가
- checkpoint 기준이 있는가

16.4 트랜잭션 의미

- 기본 격리 수준이 무엇인가
- lost update를 막는가
- phantom과 write skew를 어떻게 다룰 것인가

16.5 운영

- 슬로우 쿼리와 lock wait를 관측할 수 있는가
- backup/restore 경로가 있는가
- corruption detection 수단이 있는가

17. 핵심 정리

DBMS에서 동시성 문제를 다룰 때 가장 중요한 것은 문제 종류를 정확히 분리하는 것이다.

- 멀티스레드 문제는 공유 메모리를 어떻게 보호할 것인가
- 트랜잭션 문제는 동시에 실행된 요청의 의미를 어떻게 정의할 것인가
- 복구와 분산 문제는 장애가 나도 그 의미를 어떻게 끝까지 유지할 것인가

실무적으로는 아래 순서가 가장 안전하다.

1. coarse lock으로 correctness 확보
2. WAL과 복구 경로 정의
3. buffer pool과 page layout 고정
4. 병목을 측정한 뒤 fine-grained latch 도입
5. 필요할 때 MVCC 또는 더 높은 격리 수준 도입
6. 단일 노드가 정리된 뒤에만 replication과 consensus로 확장

즉, 좋은 DBMS는 처음부터 가장 복잡한 기법을 넣은 시스템이 아니라, 정합성을 깨지 않으면서 단계적으로 복잡성을 올릴 수 있게 설계된 시스템이다.

여기까지가 이 문서의 핵심 본문이다. 아래부터는 본문에서 언급한 주제를 더 깊게 파고드는 심화 참고 자료와, 동시성 외 설계 체크사항을 묶은 부록이다.

부록 A. 심화 주제와 기존 문서의 연결

앞에서 언급한 MVCC 내부 구조, B+Tree 동시성 제어, WAL/복구, 분산 트랜잭션은 위 내용과 직접 연결되는 심화 주제다.

즉, 새로운 별도 주제가 아니라 이 문서의 상위 개념을 코드/자료구조 수준으로 내려서 보는 확장판이다.

심화 주제	이 문서에서 연결되는 장	더 깊게 보는 대상	현재 프로젝트 우선순위
MVCC 내부 구조	6. 트랜잭션 동시성 제어, 7. 격리 수준, 9. 스토리지 엔진, 15. 프로젝트 적용	row version header, snapshot, vacuum, undo chain	중간 이후
B+Tree 동시성 제어	5. 엔진 내부 동기화, 8. 인덱스와 자료구조, 15. 프로젝트 적용	page latch, split/merge, latch coupling, B-link tree	높음
WAL/복구	10. 장애 복구와 durability, 9. 스토리지 엔진, 15. 프로젝트 적용	log record, pageLSN, checkpoint, redo/undo	가장 높음

심화 주제	이 문서에서 연결되는 장	더 깊게 보는 대상	현재 프로젝트 우선순위
분산 트랜잭션	12. 분산 DBMS, 6. 트랜잭션 동시성 제어	2PC state, participant log, consensus 연계	낮음

요약하면:

- `MVCC`는 트랜잭션 의미를 실제 row 버전 구조로 구현하는 방법이다.
- B+Tree 동시성 제어는 인덱스 구조를 여러 worker가 동시에 만질 때의 실제 latch 설계다.
- WAL/복구는 durability를 실제 로그 형식과 재시작 절차로 구현하는 방법이다.
- 분산 트랜잭션은 단일 노드 트랜잭션 의미를 여러 노드로 확장하는 방법이다.

A.1 MVCC 내부 구조

`MVCC`는 단순히 "버전을 여러 개 둔다"가 아니라, `row header`, `transaction table`, `snapshot`, `vacuum`이 함께 움직이는 구조다.

대표 자료구조 예:

```
typedef struct {
    uint64_t txn_id;
    uint64_t begin_ts;
    uint64_t commit_ts;
    int state; /* ACTIVE, COMMITTED, ABORTED */
} TxnEntry;

typedef struct {
    uint64_t xmin;      /* 이 버전을 만든 tx */
    uint64_t xmax;      /* 이 버전을 삭제/갱신한 tx, 없으면 0 */
    uint64_t undo_ptr;  /* 이전 버전 또는 undo log 위치 */
    uint32_t flags;
} RowVersionHeader;

typedef struct {
    uint64_t xmin;
    uint64_t xmax;
    uint64_t active_txns[256];
    size_t active_count;
} Snapshot;
```

읽기 시 핵심은 가시성 판단이다.

```
1. row.xmin이 아직 커밋되지 않았으면 보이지 않음
2. row.xmin이 snapshot보다 미래 tx면 보이지 않음
3. row.xmax가 0이면 살아 있는 버전
4. row.xmax가 커밋됐고 snapshot에서 이미 보이는 삭제라면 보이지 않음
5. 아니면 이 버전은 현재 트랜잭션에게 보임
```

쓰기 경로는 보통 아래처럼 흘러간다.

```
UPDATE
-> 현재 visible version 찾기
-> 새 version 생성
-> old version의 xmax 설정 또는 undo 연결
-> 인덱스 반영
-> commit 시 txn 상태 갱신
-> 나중에 vacuum이 오래된 버전 회수
```

장점:

- reader와 writer 충돌을 크게 줄인다.
- snapshot read를 자연스럽게 제공할 수 있다.

단점:

- purge/vacuum이 반드시 필요하다.
- row header와 undo chain 설계가 빈약하면 storage bloat가 빠르게 생긴다.
- 인덱스와 visibility를 함께 맞춰야 해서 구현이 어렵다.

현재 프로젝트와의 관계:

- 지금 레포는 전역 `rwlock` 기반이라 아직 MVCC가 필요하지 않다.
- 하지만 이후 `SELECT` 동시성을 높이려면 가장 먼저 검토할 수 있는 심화 주제 중 하나다.
- 다만 `WAL`, `page`, `record format`이 먼저 정리되지 않으면 MVCC를 넣어도 구조가 불안정하다.

A.2 B+Tree 동시성 제어

이 주제는 8. 인덱스와 자료구조 관점의 동시성을 실제 page/node 레벨로 내려보는 내용이다.

대표 노드 헤더 예:

```c
typedef struct {
    uint32_t is_leaf;
    uint32_t key_count;
    uint64_t page_lsn;
    uint64_t high_key;
    uint64_t right_sibling;
    pthread_rwlock_t latch;
} BTreePageHeader;
```

대표 기법:

- `search`: read latch를 잡고 부모에서 자식으로 내려간다.
- `insert`: 내려가면서 "이 노드가 split 없이 안전한가"를 검사한다.
- `safe node`가 아니면 상위 latch를 더 오래 유지하거나 split 경로로 전환한다.
- `B-link tree`에서는 `high_key`와 `right_sibling`을 이용해 concurrent split 중에도 탐색이 가능하다.

기본적인 latch coupling 흐름:

```
1. parent read latch 획득
2. child read latch 획득
3. child가 안전하면 parent latch 해제
4. leaf까지 반복
5. 쓰기면 필요한 시점에 write latch 승격 또는 재탐색
```

insert split 시 주의점:

```
1. leaf에 공간이 없으면 새 leaf 생성
2. key 절반 분할
3. sibling pointer와 high key 갱신
4. 부모 separator key 삽입
5. 필요하면 root까지 전파
```

장점:

- 전역 락 없이도 높은 인덱스 동시성을 낼 수 있다.

단점:

- split/merge와 parent update가 매우 까다롭다.
- recovery와 결합되면 pageLSN, structural modification operation까지 같이 고려해야 한다.

현재 프로젝트와의 관계:

- 지금은 `db_api.c`의 전역 `rwlock`이 B+Tree를 통째로 보호하고 있다.
- 성능을 더 올리려면 이후 `global rwlock -> table lock -> page latch` 순서로 내려가야 한다.
- 따라서 현재 레포에서 가장 직접적으로 이어지는 심화 주제 중 하나다.

A.3 WAL / 복구

이 주제는 10. 장애 복구와 durability를 실제 로그 레코드와 재시작 알고리즘 수준으로 보는 내용이다.

대표 로그 레코드 예:

```
typedef struct {
    uint64_t lsn;
    uint64_t prev_lsn;
    uint64_t txn_id;
    uint32_t type;      /* BEGIN, UPDATE, COMMIT, ABORT, CLR */
    uint64_t page_id;
    uint32_t offset;
    uint32_t before_len;
    uint32_t after_len;
} LogRecord;

typedef struct {
    uint64_t page_lsn;
    uint32_t checksum;
} PageHeader;
```

핵심 불변식:

```
page 변경을 디스크에 쓰기 전, 그 page를 설명하는 WAL record는 먼저 flush되어 있어야 한다.
```

commit 경로:

```
1. BEGIN/UPDATE log append
2. data page는 메모리 dirty 상태로만 둘 수 있음
3. COMMIT log append
4. commit lsn까지 flush
5. 그 뒤에만 클라이언트에 성공 응답
```

재시작 복구는 보통 아래 단계로 생각한다.

```
1. Analysis: 어떤 tx가 살아 있었는지, 어떤 page가 dirty였는지 파악
2. Redo: 필요한 변경을 다시 적용
3. Undo: 끝나지 않은 tx를 되돌림
```

실제로는 아래 테이블이 같이 필요하다.

```
typedef struct {
    uint64_t txn_id;
    uint64_t last_lsn;
    int state;
} TxnTableEntry;

typedef struct {
    uint64_t page_id;
    uint64_t rec_lsn;
} DirtyPageEntry;
```

장점:

- 크래시 후에도 committed change를 보존할 수 있다.
- dirty page flush를 비동기적으로 처리할 수 있다.

단점:

- 로그 포맷, LSN, checkpoint, redo/undo 순서를 정확히 맞춰야 한다.
- pageLSN 관리가 어긋나면 복구 correctness가 무너진다.

현재 프로젝트와의 관계:

- 이 레포에서 가장 먼저 깊게 들어가야 할 주제다.
- 이유는 동시성 고도화보다 먼저 `write durability`와 `crash safety`를 잡아야 하기 때문이다.
- 전역 락 기반 구조여도 WAL은 필요하다. 오히려 coarse lock 단계에서 WAL을 먼저 넣는 편이 훨씬 안전하다.

A.4 분산 트랜잭션

이 주제는 12. 분산 DBMS를 실제 coordinator/participant 상태 기계 수준으로 보는 내용이다.

대표 상태 예:

```
typedef struct {
    uint64_t txn_id;
    int state; /* INIT, PREPARING, PREPARED, COMMITTING, ABORTING, DONE */
    uint64_t participant_ids[32];
    size_t participant_count;
} CoordinatorTxn;

typedef struct {
    uint64_t txn_id;
    int state; /* INIT, PREPARED, COMMITTED, ABORTED */
    uint64_t last_lsn;
} ParticipantTxn;
```

2PC 기본 흐름:

```
1. coordinator가 PREPARE 요청 전송
2. participant는 로컬 변경을 durable하게 기록하고 PREPARED 응답
3. 모두 PREPARED면 coordinator가 COMMIT 결정
4. 일부 실패면 ABORT 결정
5. 결정은 각 participant의 로그에도 남겨야 함
```

문제점:

- coordinator가 중간에 죽으면 participant가 오래 대기할 수 있다.
- 네트워크 분할이 생기면 지연과 가용성 손실이 크다.

그래서 실무에서는:

- `2PC`만 쓰지 않고
- 메타데이터/리더 선출에는 `Raft` 같은 consensus를 함께 쓴다.

현재 프로젝트와의 관계:

- 지금 레포는 단일 노드이므로 우선순위가 가장 낮다.
- 단일 노드에서 `WAL`, `recovery`, `index concurrency`, `transaction semantics`가 정리되기 전에는 분산 트랜잭션으로 가면 안 된다.

A.5 현재 프로젝트 기준 추천 심화 순서

현재 레포 기준으로는 아래 순서가 현실적이다.

1. WAL/복구
2. B+Tree 동시성 제어
3. MVCC 내부 구조
4. 분산 트랜잭션

이 순서인 이유:

- WAL/복구가 없으면 write correctness를 보장하기 어렵다.
- 그다음에야 인덱스 내부 동시성을 세분화할 가치가 생긴다.
- `MVCC`는 storage format과 recovery 기반이 어느 정도 잡혀야 제대로 들어간다.

- 분산 트랜잭션은 단일 노드 의미가 먼저 완성되어야 한다.

부록 B. 동시성 외 추가 체크사항과 제품 사례

앞선 장들은 동시성과 트랜잭션을 중심으로 설명했지만, DBMS를 실제 제품 수준으로 만들 때는 그 외 체크사항이 더 많다.

특히 아래 항목들은 동시성과 직접 관련이 없더라도 설계 완성도와 운영 가능성을 크게 좌우한다.

B.1 추가 체크사항 요약 표

영역	무엇을 결정해야 하나	왜 중요한가	대표 채택 제품 예
저장 모델	row-store, column-store, KV, document 중 무엇인가	읽기/쓰기 패턴, 압축률, 인덱스 방식이 달라진다	PostgreSQL, MySQL InnoDB, SQLite는 row 중심 / ClickHouse는 column 중심 / MongoDB는 document 중심
온디스크 구조	page 기반인지, SSTable 기반인지, append-only인지	recovery, compaction, vacuum, update 비용이 달라진다	PostgreSQL, InnoDB, SQLite는 page 기반 / Cassandra, ScyllaDB, RocksDB, TiKV는 memtable+SSTable 계열
캐시 계층	buffer pool, shared buffer, OS cache와의 관계	메모리 사용량과 read latency를 좌우한다	InnoDB buffer pool / PostgreSQL `shared_buffers` / MongoDB WiredTiger internal cache
인덱스 전략	clustered index, secondary index, covering index, sparse index	lookup, range scan, write amplification이 달라진다	InnoDB clustered index / PostgreSQL 기본 B-tree / SQLite table/index B-tree / ClickHouse sparse primary index
로그와 복구	redo, undo, checkpoint, PITR, backup 조합	crash safety와 운영 복구 시간을 좌우한다	PostgreSQL WAL + PITR / InnoDB redo+undo+binlog / SQLite rollback journal 또는 WAL / MongoDB WiredTiger journal+checkpoint
압축/인코딩	block compression, prefix compression, column codec	저장 비용과 CPU 비용의 균형점이 달라진다	MongoDB WiredTiger compression / ClickHouse codecs / RocksDB compression
쿼리 플래너	rule-based인지, cost-based인지, 통계 기반인지	같은 SQL도 실행 성능이 크게 달라진다	PostgreSQL planner / SQLite query planner / TiDB planner / CockroachDB cost-based optimizer
실행 엔진	row-at-a-time, iterator, vectorized, distributed execution	OLTP/OLAP 성능 특성이 크게 갈린다	SQLite/전통 RDBMS iterator 계열 / ClickHouse vectorized / CockroachDB DistSQL
스키마 변경	online DDL, concurrent index build, declarative schema change	운영 중 스키마 변경 가능성이 달라진다	MySQL InnoDB Online DDL / PostgreSQL concurrent index build / CockroachDB declarative schema changes
백업/복구 운영	full backup, incremental, PITR, restore drill	장애 시 실제 복구 가능 여부를 결정한다	PostgreSQL base backup + WAL archive / MySQL full backup + binary log PITR / CockroachDB scheduled backup
관측성	query stats, lock wait, slow query, internal metrics	병목과 장애 원인을 찾아낼 수 있어야 한다	PostgreSQL `pg_stat_statements` / MySQL Performance Schema / CockroachDB metrics
보안	인증, 권한, TLS, at-rest encryption, audit	제품화 단계에서 빠질 수 없는 요소다	PostgreSQL roles + TLS / MySQL TLS + InnoDB encryption / MongoDB auth + TLS
데이터 수명 주기	TTL, compaction, vacuum, archive, cold storage	오래된 데이터 처리 정책이 없으면 비용이 폭증한다	PostgreSQL VACUUM / TiDB GC / ClickHouse TTL / Cassandra compaction+TTL

영역	무엇을 결정해야 하나	왜 중요한가	대표 채택 제품 예
API/호환성	SQL dialect, wire protocol, client driver 호환성	도입성과 생태계를 크게 좌우한다	TiDB는 MySQL 호환 지향 / CockroachDB는 PostgreSQL wire 호환 지향 / SQLite는 embedded C API

B.2 체크사항별 설명

B.2.1 저장 모델

DBMS는 먼저 무엇을 기본 단위로 저장할 것인가를 정해야 한다.

- row-store: 한 row의 컬럼들이 같이 저장된다.
- column-store: 같은 컬럼 값들을 묶어 저장한다.
- document-store: JSON/BSON 같은 문서 단위를 기본으로 둔다.
- KV-store: key/value 자체를 기본 추상화로 둔다.

적합한 곳:

- row-store: OLTP
- column-store: 집계/분석/스캔
- document: 유연한 스키마
- KV: 저수준 저장 엔진, 메타데이터 저장소

대표 제품:

- PostgreSQL, MySQL InnoDB, SQLite: row 중심
- ClickHouse: column 중심
- MongoDB: document 중심
- RocksDB, TiKV: KV 중심

B.2.2 온디스크 구조

같은 row-store라도 실제 디스크 구조는 다를 수 있다.

- page 기반: in-place update, buffer pool, page split, WAL과 잘 맞는다
- SSTable 기반: immutable file + compaction, 쓰기 집약적 workloads에 강하다
- append-only: 구현 단순성과 복구 단순성을 얻을 수 있다

대표 제품:

- PostgreSQL, InnoDB, SQLite: page 기반
- Cassandra, ScyllaDB: memtable + commit log + SSTable
- TiKV: RocksDB 기반 LSM + Raft 복제

B.2.3 캐시와 메모리 계층

메모리를 어떻게 쓰는지는 성능과 운영성을 동시에 좌우한다.

체크 포인트:

- DB 내부 캐시를 둘 것인가
- OS page cache와 중복될 때 정책을 어떻게 둘 것인가
- warm-up 시간을 줄이는 장치를 둘 것인가

대표 제품:

- MySQL InnoDB: `buffer pool`
- PostgreSQL: `shared_buffers`
- MongoDB WiredTiger: internal cache + filesystem cache 병행

주의:

- 캐시를 크게 잡는 것이 항상 좋은 것은 아니다.
- flush 정책과 checkpoint 정책이 따라오지 않으면 오히려 stall이 심해질 수 있다.

B.2.4 인덱스 전략

인덱스는 "있냐 없냐"보다 어떤 구조와 유지 비용을 감수할 것인가가 더 중요하다.

대표 선택:

- clustered index
- secondary index
- covering index
- sparse/partial index
- data-skipping index

대표 제품:

- InnoDB: primary key를 중심으로 한 clustered index
- PostgreSQL: 기본 B-tree, 필요에 따라 GIN/GiST/BRIN
- SQLite: table/index 모두 B-tree 기반
- ClickHouse: sparse primary index + data skipping index

B.2.5 로그, 백업, 복구

crash recovery와 운영 backup은 같이 설계해야 한다.

체크 포인트:

- WAL/redo만 있는가, undo도 있는가
- PITR이 가능한가

- backup 중에도 서비스가 계속 가능한가
- restore 시간을 예측할 수 있는가

대표 제품:

- PostgreSQL: WAL + checkpoint + continuous archiving + PITR
- MySQL InnoDB: redo/undo + binary log 기반 PITR
- SQLite: rollback journal 또는 WAL mode
- MongoDB WiredTiger: journal + checkpoint

B.2.6 압축과 인코딩

대규모 데이터에서는 압축이 선택이 아니라 사실상 기본 기능이다.

체크 포인트:

- row/page 단위 압축인지
- index prefix compression을 지원하는지
- column codec을 둘 것인지
- 압축률과 CPU 비용의 균형이 어떤지

대표 제품:

- MongoDB WiredTiger: collection/index/journal 압축
- ClickHouse: LZ4, ZSTD 등 column codec 활용
- RocksDB 계열: block compression 사용 가능

B.2.7 쿼리 플래너와 실행 엔진

DBMS는 데이터를 저장하는 것보다 어떻게 읽을지 결정하는 계층이 더 어려운 경우가 많다.

체크 포인트:

- 통계 수집이 있는가
- cost model이 있는가
- index scan vs table scan을 고를 수 있는가
- sort, join, aggregation이 어떤 연산 모델을 쓰는가

대표 제품:

- SQLite: 가벼운 query planner
- PostgreSQL: 전통적인 cost-based optimizer
- ClickHouse: columnar/vectorized execution에 최적화
- CockroachDB: 분산 SQL 실행과 cost-based optimizer

B.2.8 스키마 변경

실제 서비스에서는 스키마 변경을 얼마나 안전하게 할 수 있는지가 매우 중요하다.

체크 포인트:

- index 생성 중 서비스 지속 가능 여부
- table rewrite가 필요한지
- schema migration이 트랜잭션과 충돌할 때 정책이 무엇인지

대표 제품:

- MySQL InnoDB: Online DDL
- PostgreSQL: concurrent index build
- CockroachDB: declarative schema change 계열

B.2.9 관측성과 운영 도구

운영에서 가장 중요한 것은 "왜 느린지"를 바로 볼 수 있는가다.

최소 요구:

- slow query log
- query digest/statement stats
- lock wait/contended query 통계
- cache hit ratio
- checkpoint/compaction backlog
- backup 상태

대표 제품:

- PostgreSQL: `pg_stat_statements`
- MySQL: Performance Schema
- CockroachDB: SQL/cluster metrics

B.2.10 데이터 수명주기 관리

데이터는 쌓기만 하면 끝이 아니라, 언제 지우고, 언제 압축하고, 언제 옮길지가 필요하다.

대표 정책:

- vacuum
- MVCC GC
- compaction
- TTL

- archive/cold storage

대표 제품:

- PostgreSQL: VACUUM / autovacuum
- TiDB/TiKV: MVCC GC
- ClickHouse: TTL
- Cassandra: TTL + compaction

B.3 제품별 대표 설계 조합

제품	대표 설계 조합	이 문서와 연결되는 핵심 포인트
PostgreSQL	row-store, page 기반, MVCC, WAL, B-tree 기본 인덱스, `shared_buffers`, PITR	범용 RDBMS의 전형적인 균형형 설계
MySQL InnoDB	clustered B+Tree, MVCC, redo/undo, buffer pool, next-key lock, binary log	OLTP 중심 설계와 강한 운영 기능
SQLite	embedded, 단일 파일, B-tree, rollback journal/WAL, 가벼운 planner	단순성, 내장형, 낮은 운영 복잡도
CockroachDB	distributed SQL, MVCC, Raft, distributed transaction, cluster metrics	강한 일관성과 분산 확장을 동시에 추구
TiDB/TiKV	SQL layer + TiKV, RocksDB 기반 LSM, MVCC, Raft, Percolator 계열 transaction	MySQL 호환 분산 HTAP/OLTP 계열
Cassandra	memtable + SSTable, commit log, compaction, TTL, 분산 확장 중심	쓰기 집약적 분산 저장소 설계
ScyllaDB	Cassandra 계열 SSTable 구조, shard-per-core 최적화, compaction	고성능 분산 쓰기/읽기 지향
ClickHouse	column-store, MergeTree, sparse primary index, background merge, codecs	분석/집계 최적화, 높은 압축 효율
MongoDB + WiredTiger	document model, MVCC, optimistic concurrency, journal+checkpoint, compression	문서 지향 + 범용 운영 기능의 절충형
RocksDB	embedded KV, memtable + SST, compaction, compression	다른 DBMS의 저장 엔진으로 자주 채택됨

B.4 현재 프로젝트 기준으로 우선 추가해야 할 비동시성 체크사항

이 레포에서는 동시성 개선 전에 아래 항목들을 먼저 문서화하거나 구현하는 편이 좋다.

1. 파일 포맷 버전: 앞으로 row/page 구조가 바뀌어도 복구 가능하도록 버전 필드가 필요하다.
2. write durability 규칙: 어떤 시점에 성공 응답을 보낼지, 현재 파일 쓰기 경로의 원자성 규칙이 필요하다.
3. 백업/복원 절차: 최소한 DB 디렉터리 백업과 복원 순서를 문서로 정해야 한다.
4. 관측성: queue 길이, worker 처리량, SQL latency, error count를 바로 볼 수 있어야 한다.
5. 인덱스/스토리지 형식 고정: 지금의 B+Tree와 row 저장 형식을 향후 WAL과 연결할 수 있게 명세화해야 한다.

6 테스트 데이터 무결성 검사: restart 후 index와 data 파일이 일치하는지 검사하는 루틴이 필요하다.

정리하면, 동시성은 중요하지만 제품 수준 DBMS 설계는 저장 구조, 복구, 운영, 관측, 백업, 압축, 호환성까지 함께 잡아야 한다. 실제 유명 제품들도 한 가지 기법만으로 성공한 것이 아니라, 이 여러 층위를 일관되게 묶어서 설계했다.